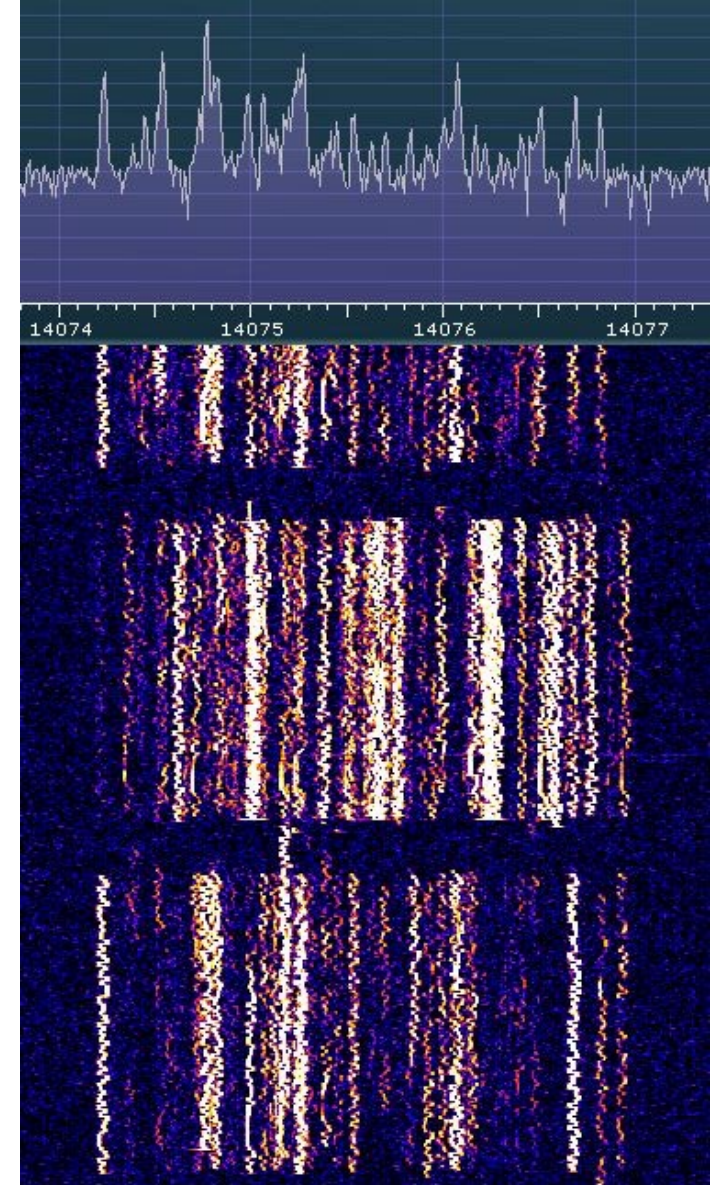


Pushing FT8 Decoding Further

Weak signals, soft evidence, and faster decoders

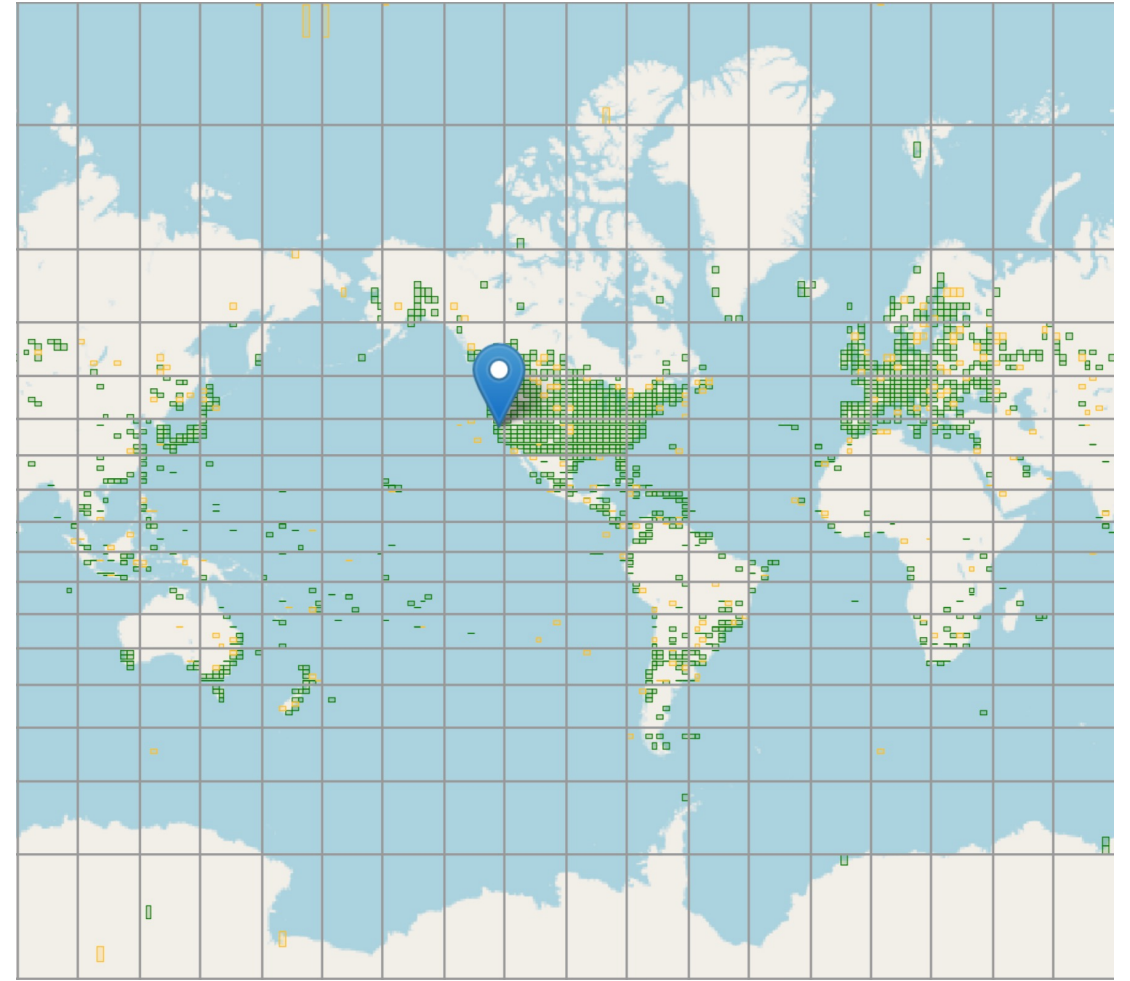
David Christle, KD0BTO

PAARA, May 2026



Who Am I

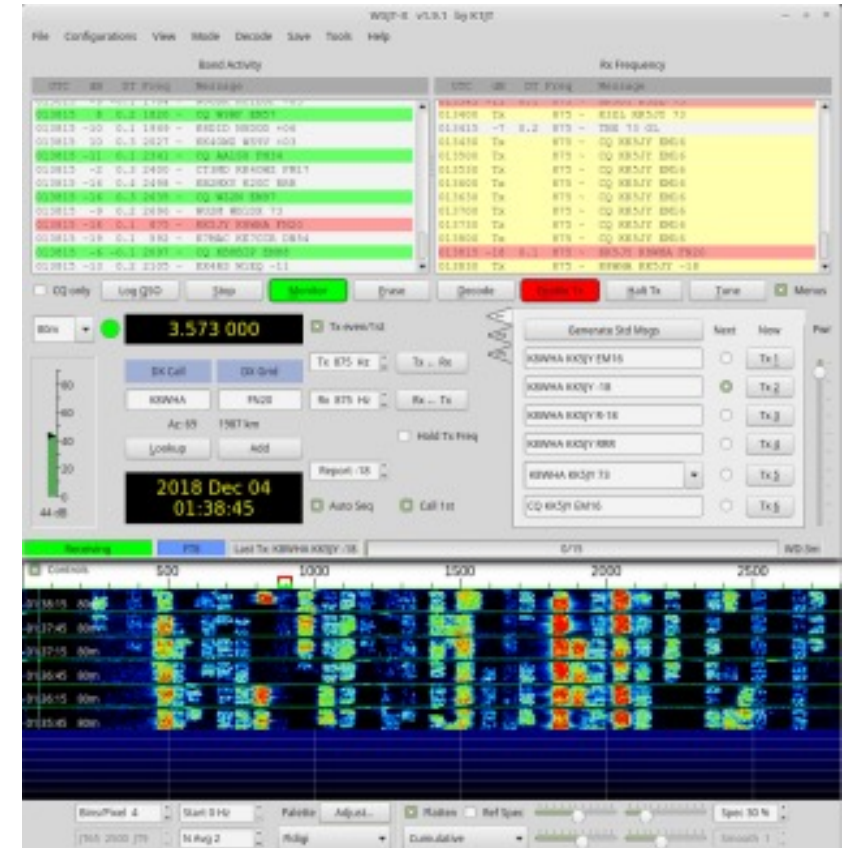
- David Christle, KD0BTO
- Licensed in college at the Univ. of Minnesota in 2007 (General)
- Returned seriously to HF in 2024 (Extra)
- Physics PhD: experimental quantum computing, lasers, microwaves, statistical data analysis
- Now work in machine learning and data infrastructure
- The original appeal of amateur radio for me is the sense of wonder it works at all: modest power, no wires, no giant dishes, just signals bending through the ionosphere and reaching the other side of the world
- DXpeditions, JT65, and FT8 all push that idea toward the limit
- `ft8lib` started as a Rust learning project and became a way to study FT8 decoding seriously



KD0BTO Grid Squares Worked

What is this talk about?

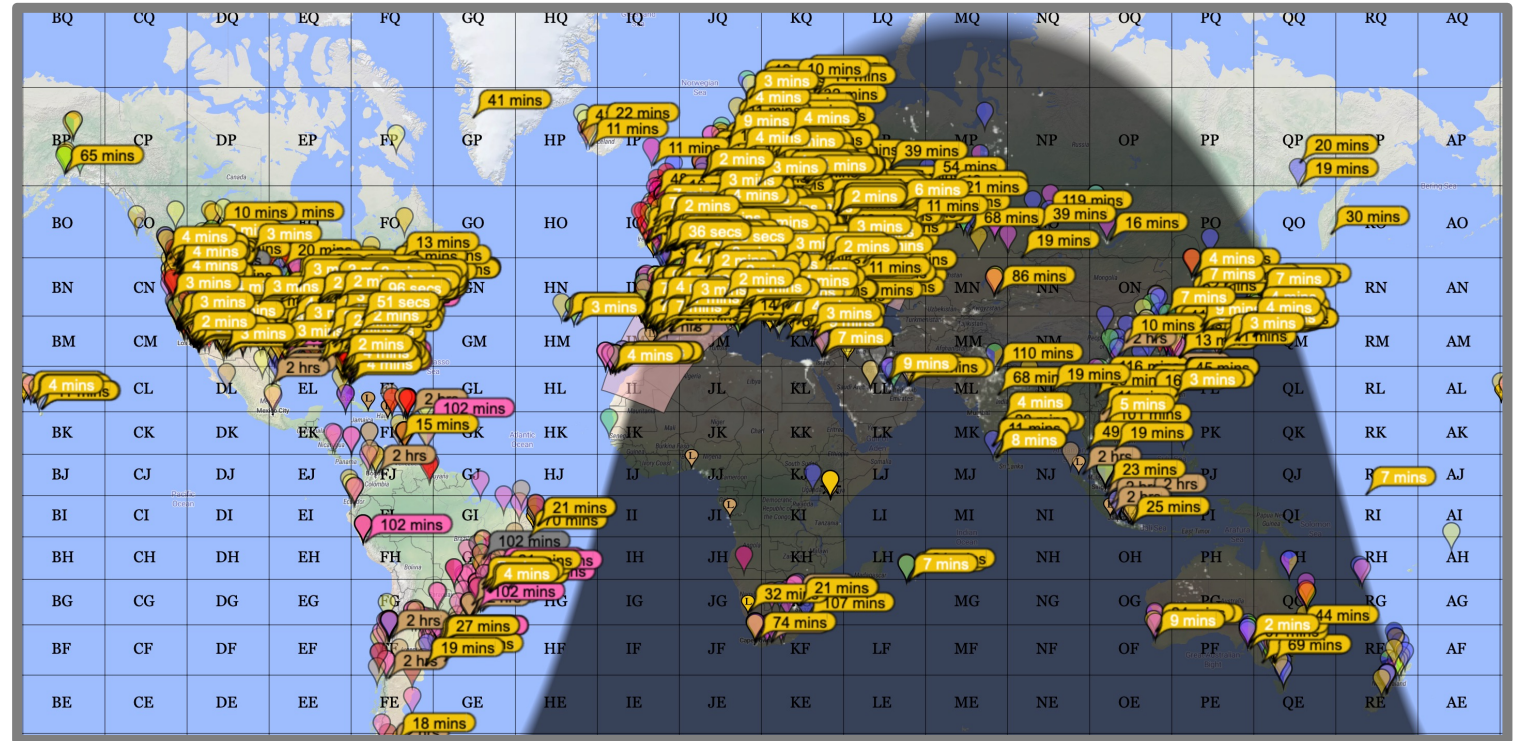
- FT8 looks simple: every 15 seconds, a few decoded lines appear.
- The decoder is doing a lot of work underneath: finding signals, estimating bit confidence, repairing errors, checking CRCs, subtracting decoded signals, and trying again.
- **ft8lib** is my Rust implementation of that pipeline: a way to understand WSJT-X closely, and a testbed for pushing weak-signal decoding.
- “Pushing further” here means two things:
 - **Recover more weak signals** when we can afford deeper decoding
 - **Use less CPU** when we want FT8 on smaller devices, skimmers, SDRs, or portable stations
- Constraints:
 - Only 2.4 seconds to respond – cycles start every 15s in FT8
 - Extra decodes only count if the false-decode rate stays low.



WSJT-X Interface

Weak-signal communication in amateur radio

- **DX:** Weak-signal modes make long-path, low-angle, marginal propagation contacts practical
- **POTA / SOTA:** Activators often use small radios, battery power, temporary antennas, and imperfect locations
- **QRP (low power):** When you cannot raise power, improve the mode, coding, and decoding instead
- **Compromised home stations:** HOAs, small lots, urban noise, indoor antennas, RFI limits, and lossy installs



Andy 5Z4VJ – Nairobi, Kenya 05/01/2026

100W + a Hex-Beam/wire vertical

If you can't hear 'em, you can't work em!

"Station is located in a city apartment with high, intermittent local noise levels, so please be patient."

Weak-signal communication in amateur radio

- **DX:** Weak-signal modes make long-path, low-angle, marginal propagation contacts practical
- **POTA / SOTA:** Activators often use small radios, battery power, temporary antennas, and imperfect locations
- **QRP (low power):** When you cannot raise power, improve the mode, coding, and decoding instead
- **Compromised home stations:** HOAs, small lots, urban noise, indoor antennas, RFI limits, and lossy installs



Andy 5Z4VJ – Nairobi, Kenya 05/01/2026

100W + a Hex-Beam/wire vertical

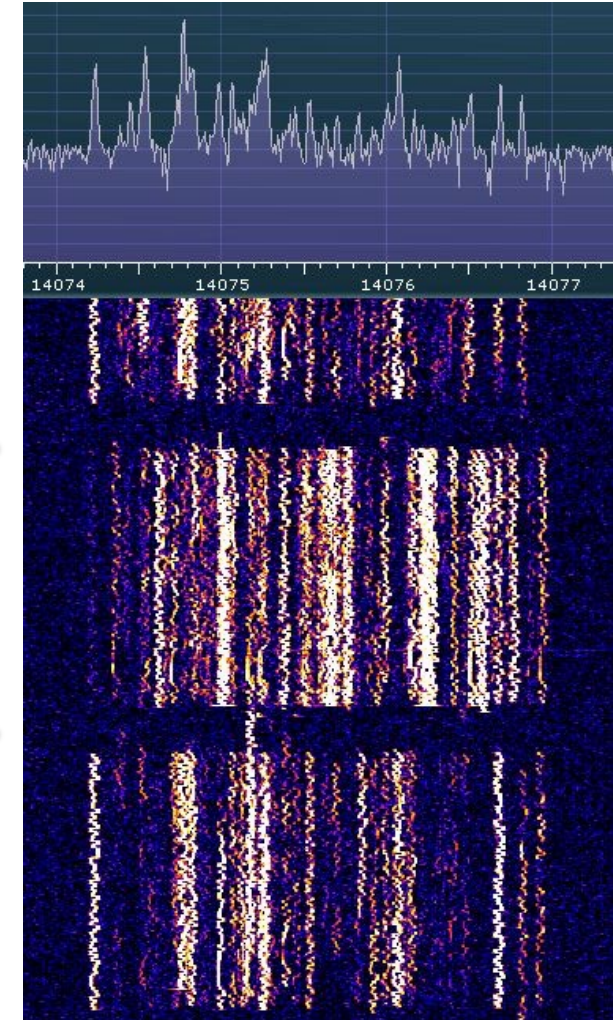
If you can't hear 'em, you can't work em!

"Station is located in a city apartment with high, intermittent local noise levels, so please be patient."

What is FT8?

- Released in 2017 as part of WSJTX program by K1JT & team
- FT8 Creators: Steve Franke, K9AN + Joe Taylor, K1JT
- FT8: “Franke-Taylor design, 8-FSK”
 - Frequency Shift Keying with 8 tones (aka symbols)
 - Very narrow bandwidth ~55 Hz
 - 15 seconds per transmit/receive cycle
- Optimized for **weak HF ionospheric operation**, especially low SNR, moderate Doppler/frequency spread, fading, and crowded passbands.

15 seconds



Average Day on 20m FT8
14.074 MHz (3 kHz bandwidth)

What Does a Standard FT8 Exchange Look Like?

CQ K1ABC FN42

K1ABC W9XYZ EN37

W9XYZ K1ABC -22

K1ABC W9XYZ R-19

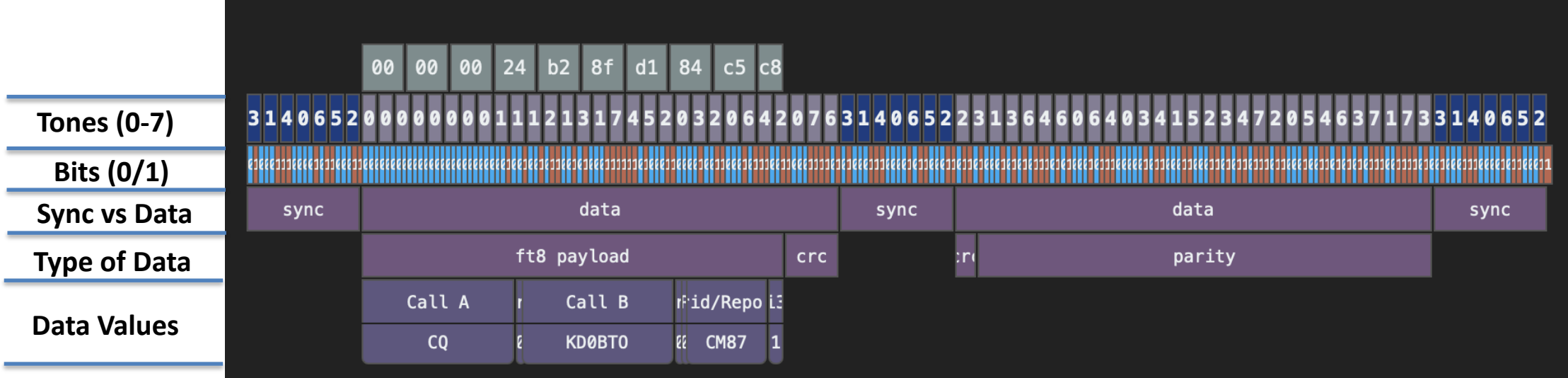
W9XYZ K1ABC RRR

K1ABC W9XYZ 73

- Structured messages, not chat
- Similar to contest exchanges
- Callsigns + Grid Squares + Signal Reports + RR73/73
- *Limiting the kinds of messages that can be exchanged means the messages can be **encoded** into a fewer number of bits (more densely packed, aka **more efficient**)*

From Message To Bits To FSK Tones

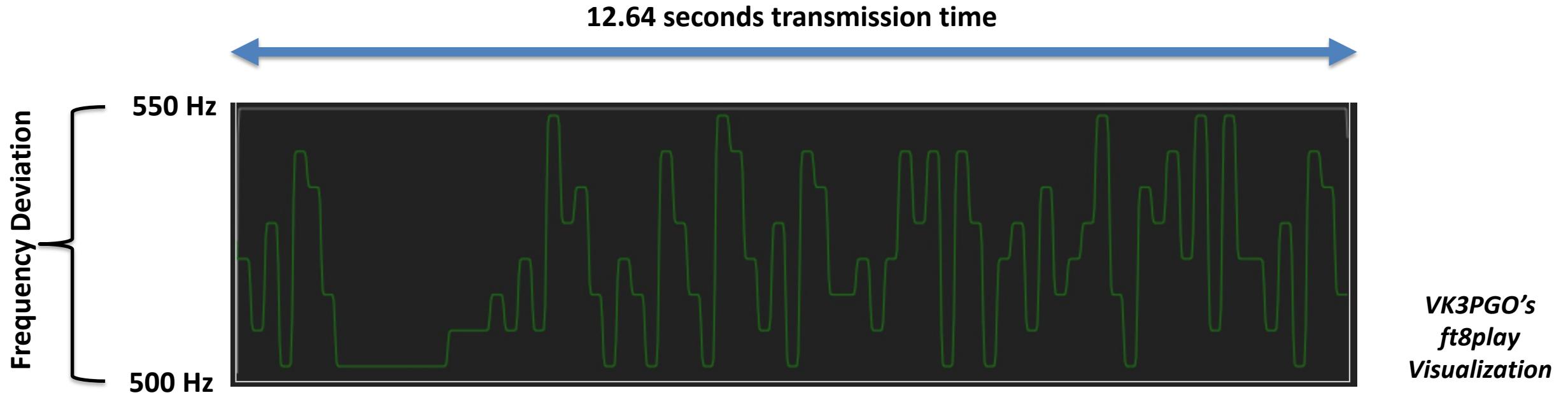
How “CQ KD0BTO CM87” Is Encoded Into Bits & FSK Tones



VK3PGO's
ft8play
Visualization

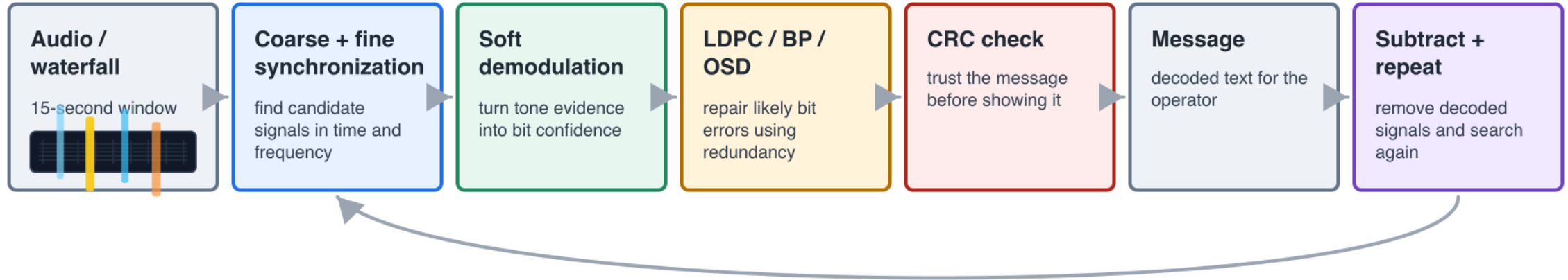
- `CQ KD0BTO CM87` is not stored as ordinary text
- FT8 packs the radio-contact meaning into **77 payload bits**
- Add **14 CRC bits** to check the answer + Add **83 LDPC parity bits** to repair errors
- Help receiver find our signal: **7 Sync Tones** (Costas array), always the same, at beginning + middle + end
- Result: **174 coded bits** for the receiver to work with -> **79 FSK Tones (Symbols) Transmitted Over The Air**

Transmission Over The Air



- 58 data symbols carry the coded bits
- $3 \times 7 = 21$ sync symbols are fixed Costas Array Tone patterns (beginning/middle/end)
- 1 symbol = 0.16 seconds, so 79 total symbols = 12.64 seconds
- 15.0 second transmit period of FT8 → entire decoding process must finish in 2.36s

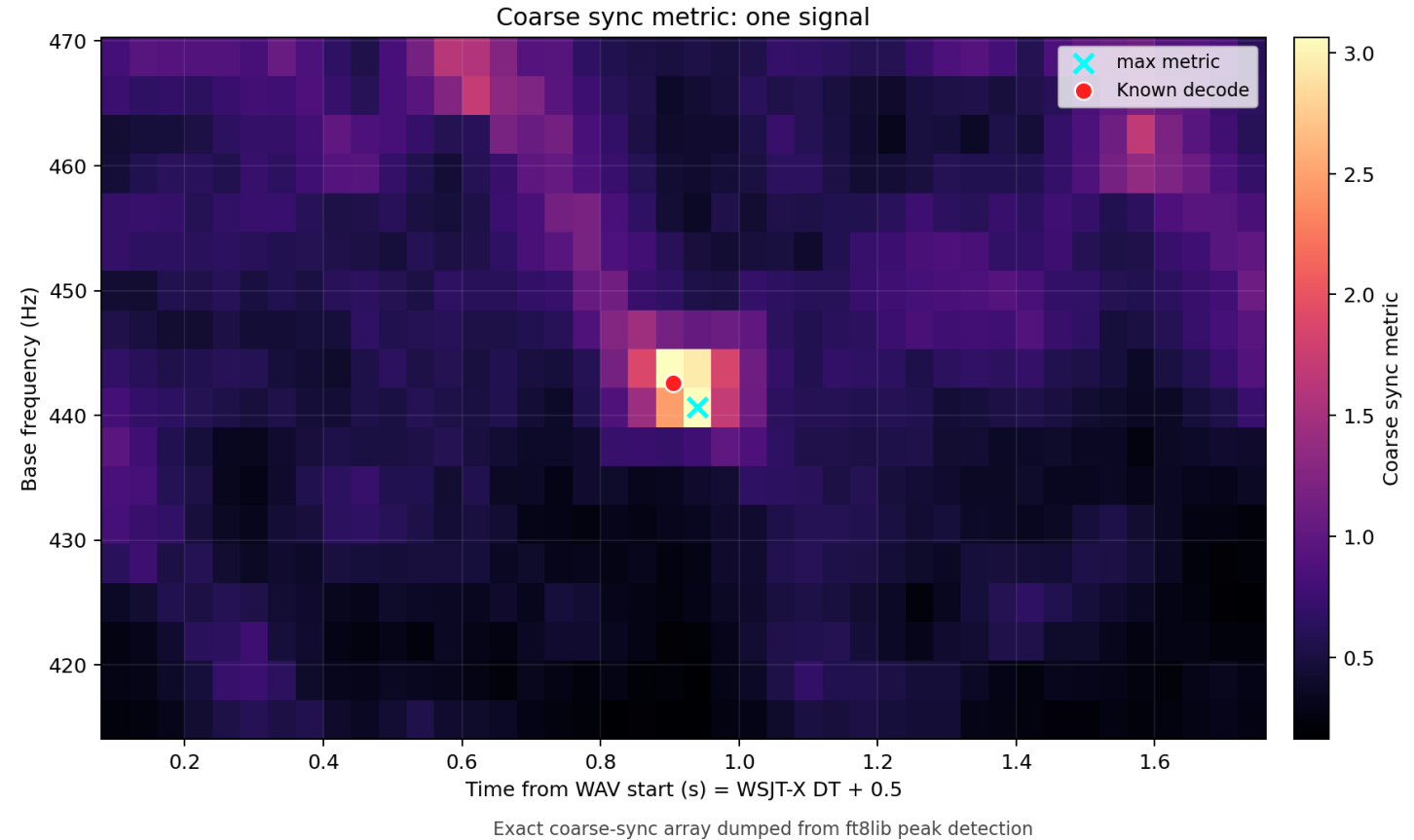
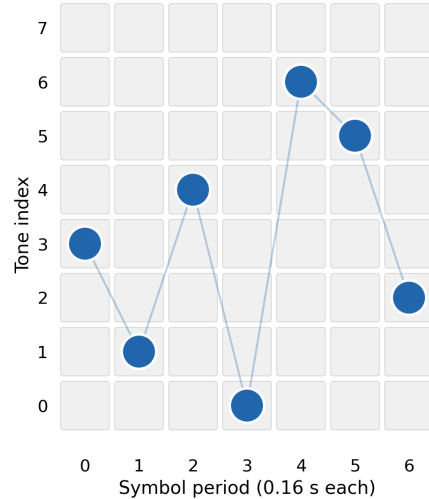
Now We Go In Reverse: The FT8 Receive Chain



- **Coarse + fine synchronization**: find candidate signals in time and frequency using Costas Array Tones
- **Soft demodulation**: figure out “which FSK tone did we hear?” every 0.16s symbol, turn tone evidence into bit probabilities (“log likelihood ratio” LLR – quantify the probability of 0 vs 1)
- **LDPC error correction**: repair likely bit errors
- **CRC check**: trust the message before showing it
- **Subtract + repeat**: remove decoded signals and search again

How do we find potential signals of unknown time & frequency?

FT8 Costas Synchronization Pattern [3, 1, 4, 0, 6, 5, 2]



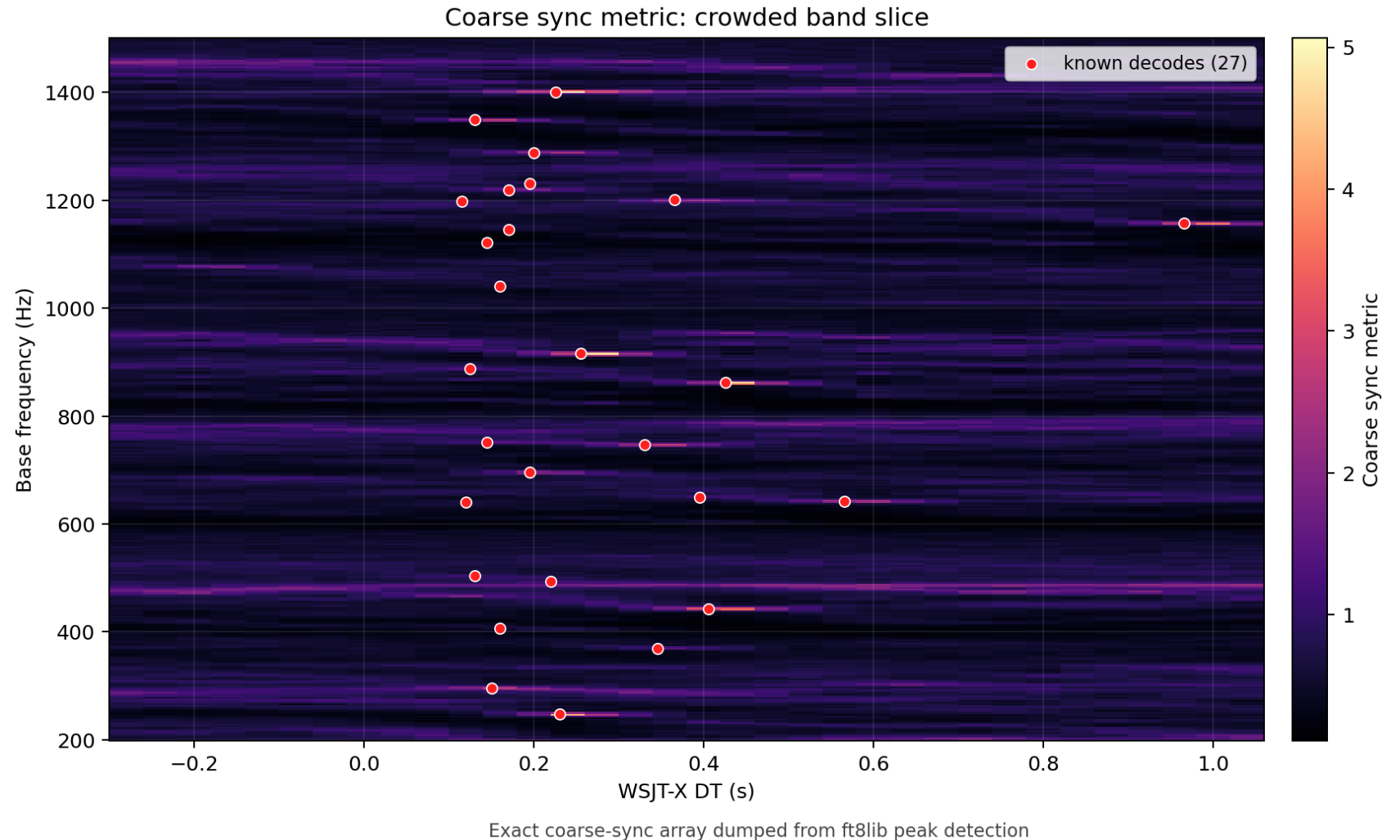
- Receiver does not know exact time nor frequency
- But must be accurate to within $\sim 15\text{ms}$ / $\sim 1\text{Hz}$
- Slide the known Costas template across time and frequency -- creates a "Sync Score" in 2D:

Frequency vs Start Time

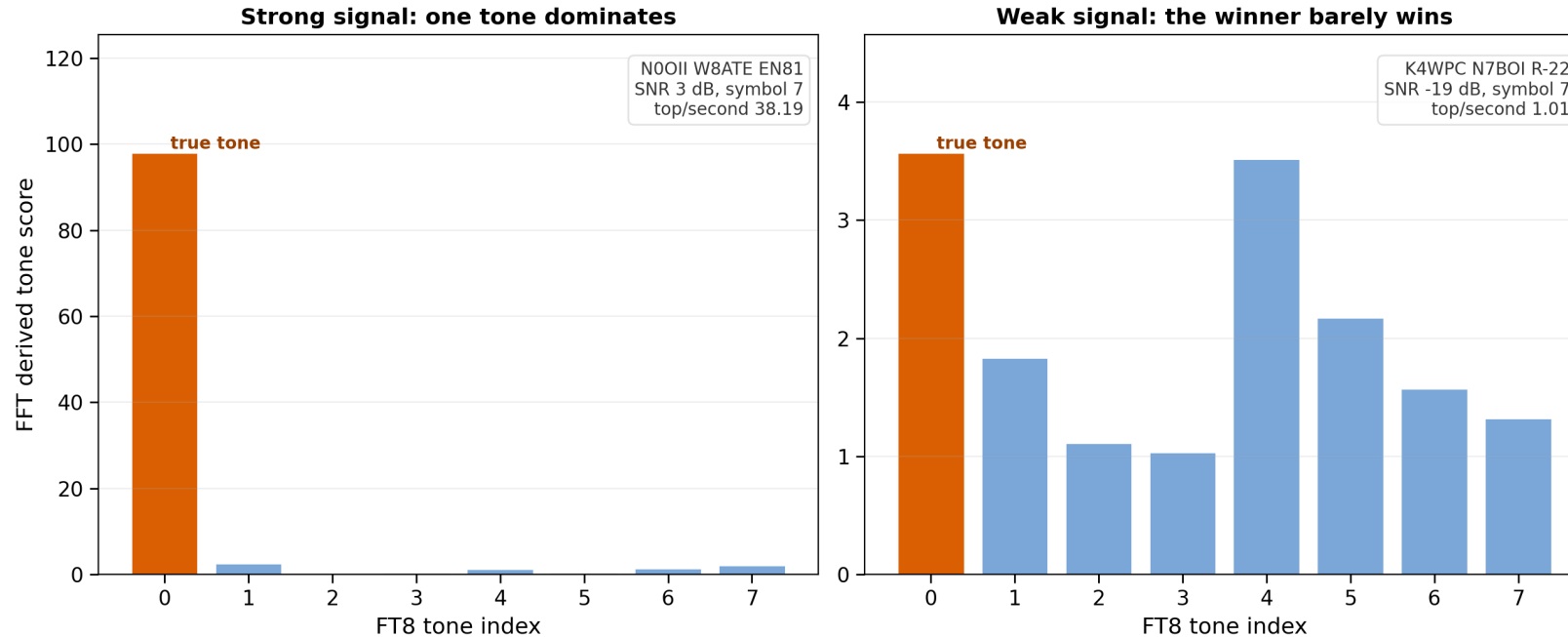
- Peaks become candidate FT8 signals

Synchronization Score in a Crowded FT8 Slot

- Zoom out from one sync peak to the band slice (200-1500 Hz)
- Red dots are known successful decodes
- Signals share the same 15-second window
- They differ mainly by audio frequency and small timing offsets
- **The synchronizer is building a candidate list**



Frequency Spectrum of One Tone in Strong vs Weak Conditions

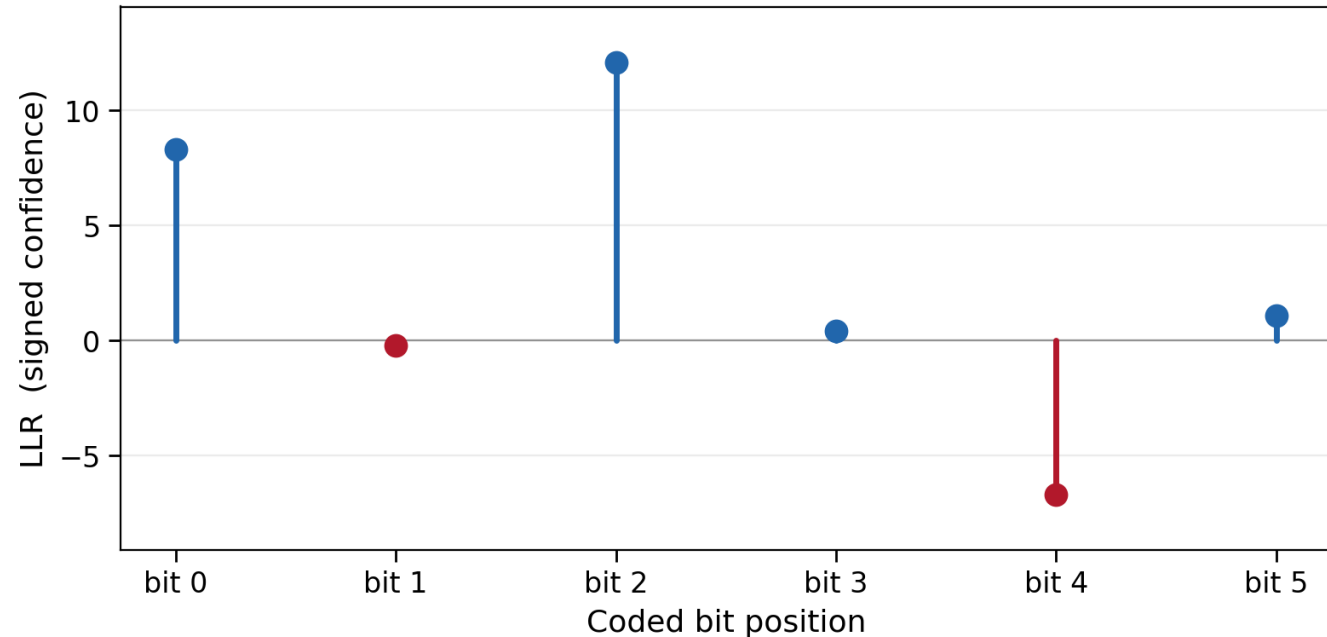


- Each symbol chooses one of eight tones
- An FFT slice gives evidence for all eight (i.e. ‘what is the energy at each tone frequency?’)
- **Largest energy bin is not always correct** – we’re in the **weak-signal** regime, after all – **so don’t throw that info away!**
- Note for later: Here, we’re keeping it simple by only looking at the frequency bins of tones. But energy at off-tone frequencies has information, too, e.g. about the local noise environment

Hard Decisions vs Soft Evidence (Log Likelihood Ratios)

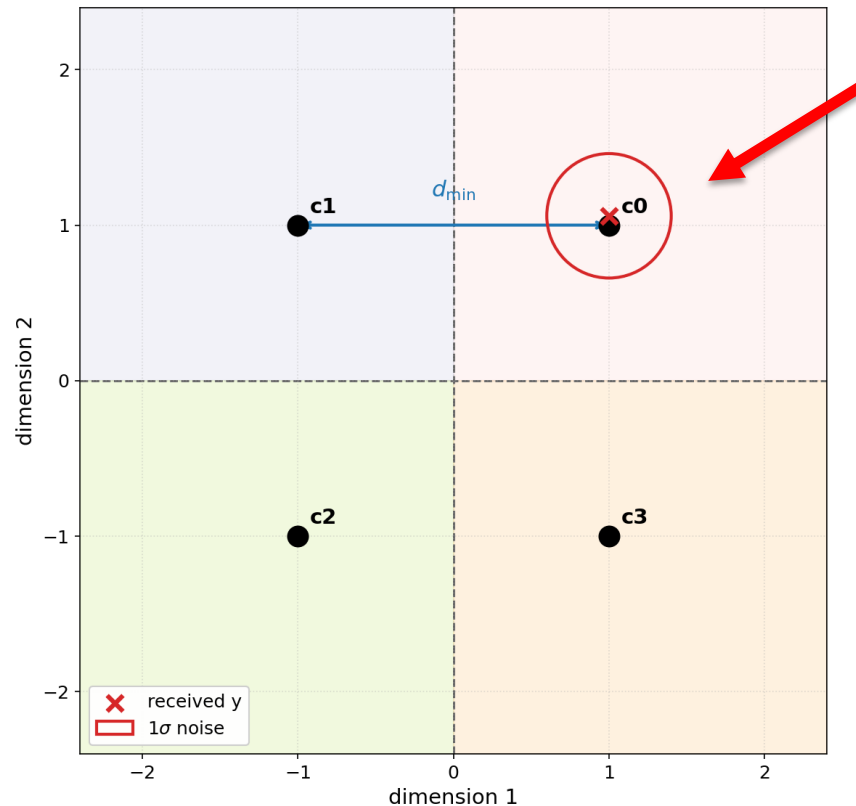
Hard bits:	1	0	1	1	0	1
Soft values:	+8.3	-0.2	+12.1	+0.4	-6.7	+1.1

- Hard decision: pick the winner and treat it as certain
- Soft evidence: keep the confidence attached to the guess
- Quantified by "log likelihood ratio" (LLR)
- *More positive value → more likely to be 1*
- *More negative value → more likely to be 0*
- *A value of 0.0 is a 50/50 chance*



Basic Gist of How Error Correction Works

No Error Correction (Raw Bits)



With **low** noise, the “LLR” estimate is **very close** to a valid value

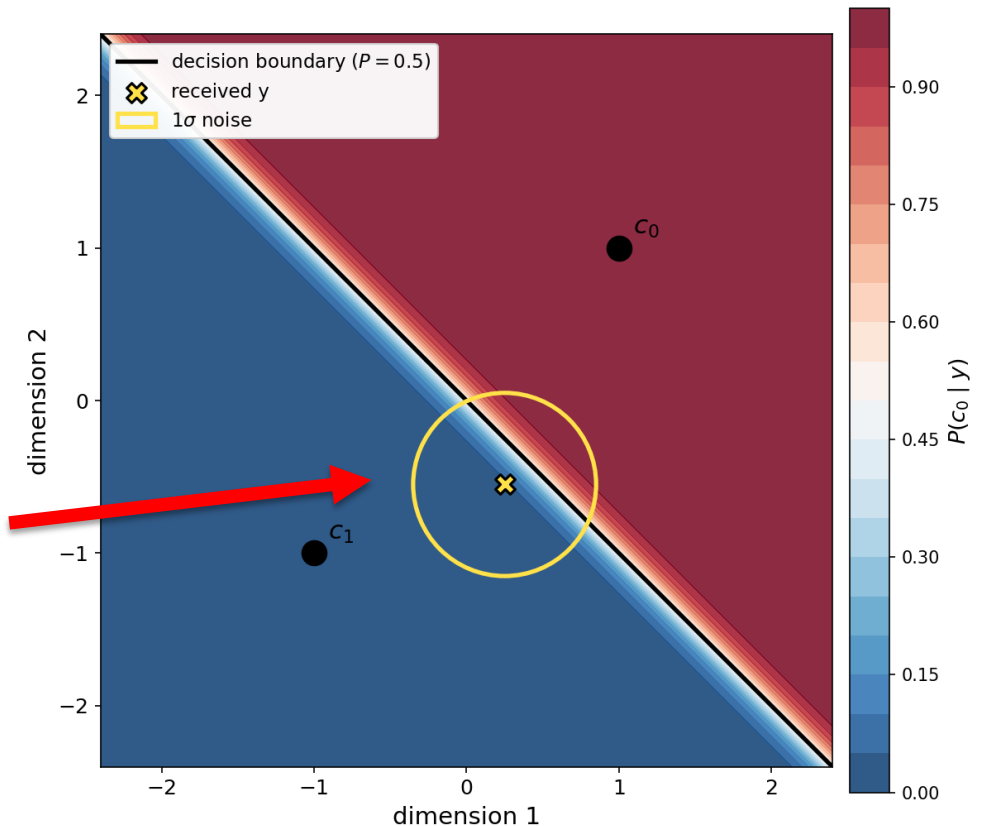
With **high** noise, the “LLR” estimate is **further away** from a valid value.

High uncertainty!

All 4 values allowed: $(-1, +1)$, $(-1, -1)$, $(+1, -1)$, $(+1, +1)$

Minimum distance between two valid values: 1.0

With Error Correction Code



Only specific values allowed: $(-1, -1)$, $(+1, +1)$

Minimum distance between two valid values: 1.414

Larger **distance** between valid values → **More noise** is required to cause a bit flip

Error Correction in FT8: Low Density Parity Check Codes

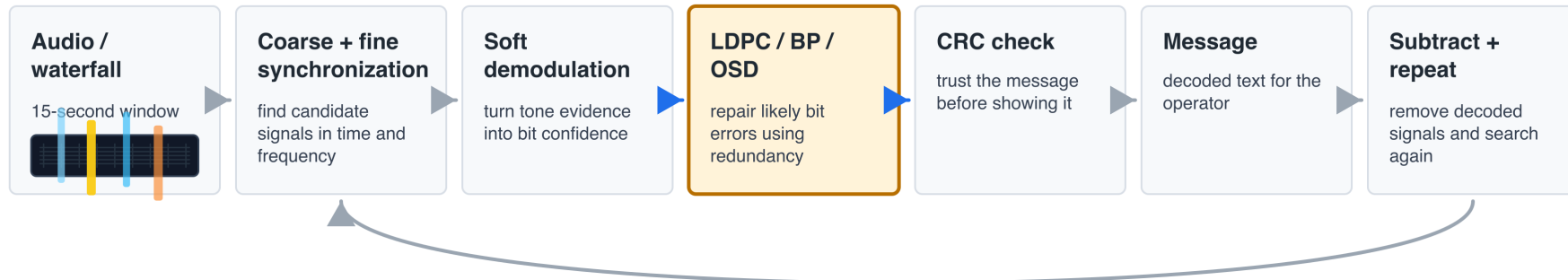
- FT8 uses a Low Density Parity Check (LDPC) code
- First conceived in 1960 by Robert Gallager, along with an efficient decoder algorithm -- essentially what we know as “Belief Propagation” (**BP**) today
- No computers fast enough in 60’s, so largely forgotten.
- 1990’s: Rediscovered when MacKay/Neal proved they achieve close to the “Shannon limit” (aka the theoretical best for *any* error correction code) + patent-free
- **Input:** Data in Bits, **Output:** Data with Extra “Parity” Bits

Key methods used for decoding LDPC codes & validation:

Belief Propagation (BP) : fast iterative repair across parity checks

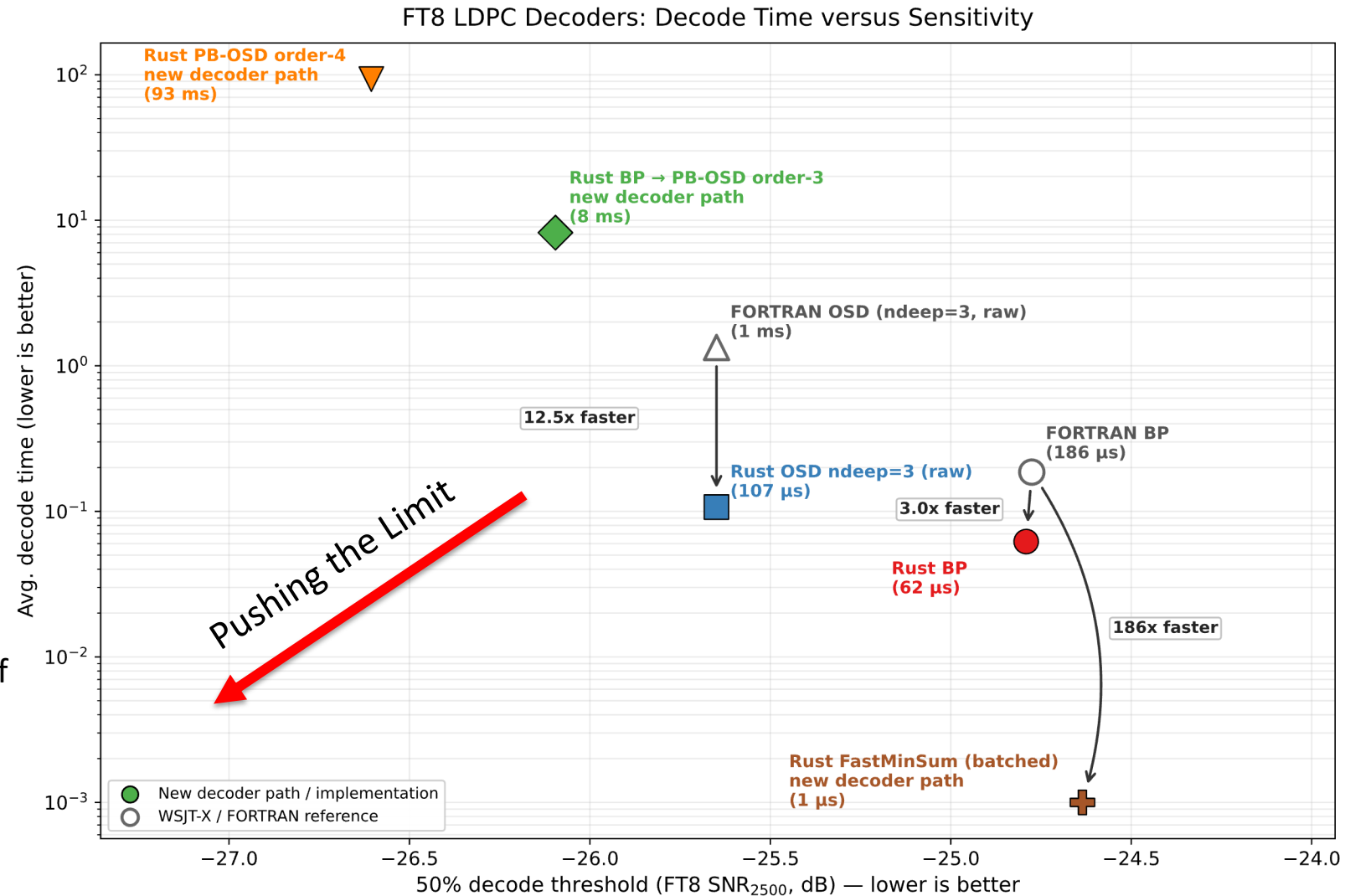
Order Statistic Decoder (OSD): deeper repair search guided by confidence

Cyclic Redundancy Check (CRC): error detection only – not correction. Final trust check before showing the message

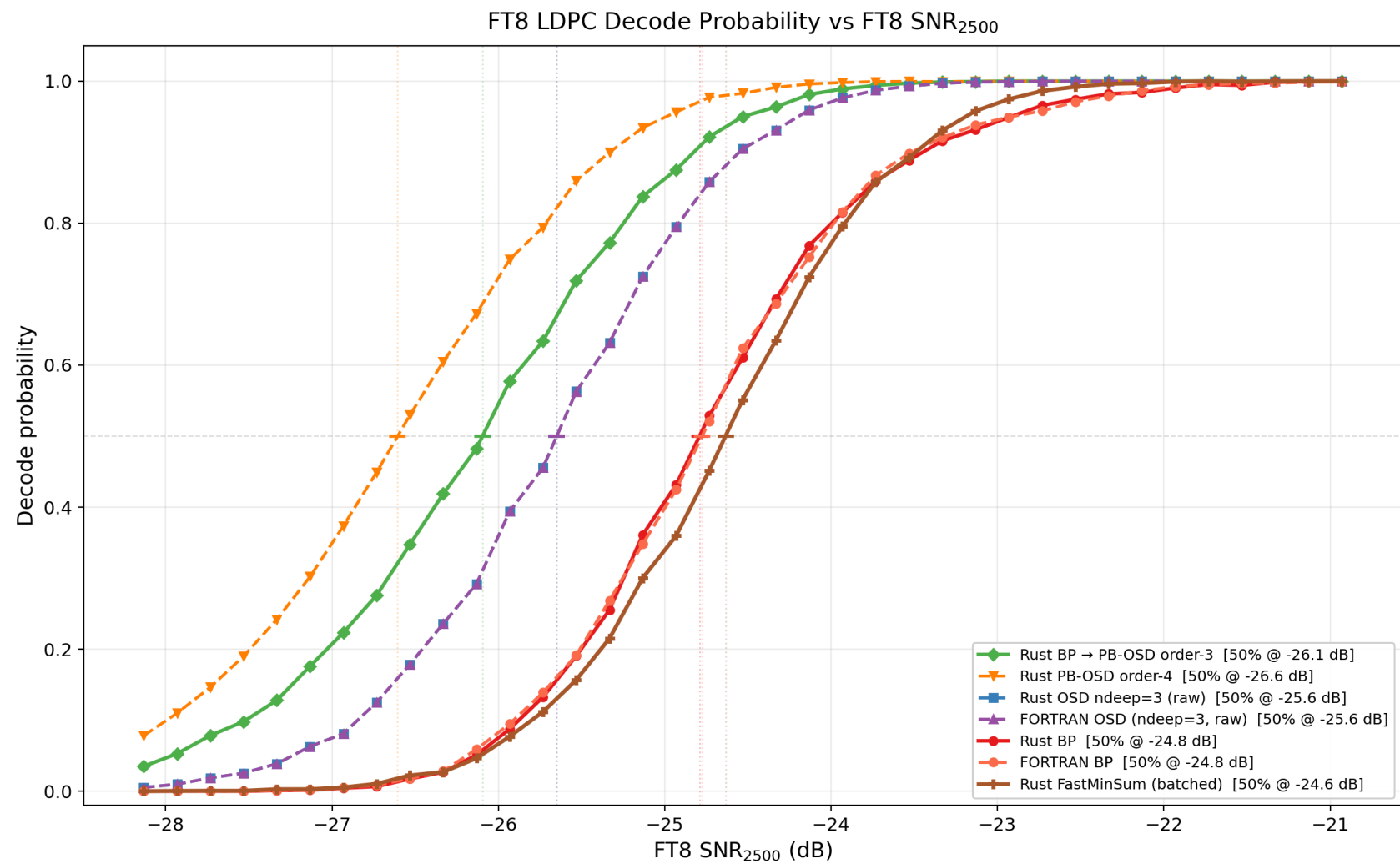


New Decoder Implementations for Better Sensitivity vs CPU Time Tradeoffs

- **Tradeoff:** More sophisticated decoders, testing more possible “bit flips”, means more decodes. But deeper decoding is slower.
- New “**PB-OSD**” and “**Fast-Min-Sum**” decoders, **new in Rust library**. They are best decoders at the low & high ends of the sensitivity vs decode time curve.
- Also made existing WSJT-X decoders faster – “**Belief Propagation (BP)**” and “**OSD**” – ft8lib library versions are **3x** and **12x** faster
- **Faster decoders let us run *better* error correction with same CPU** – improve changes of decode on computers with weaker CPUs & improve sensitivity on high-powered desktops
- **Not limited to using one: try high SNR candidates with *fast* decoder and save the *slow* decoders for low SNR**

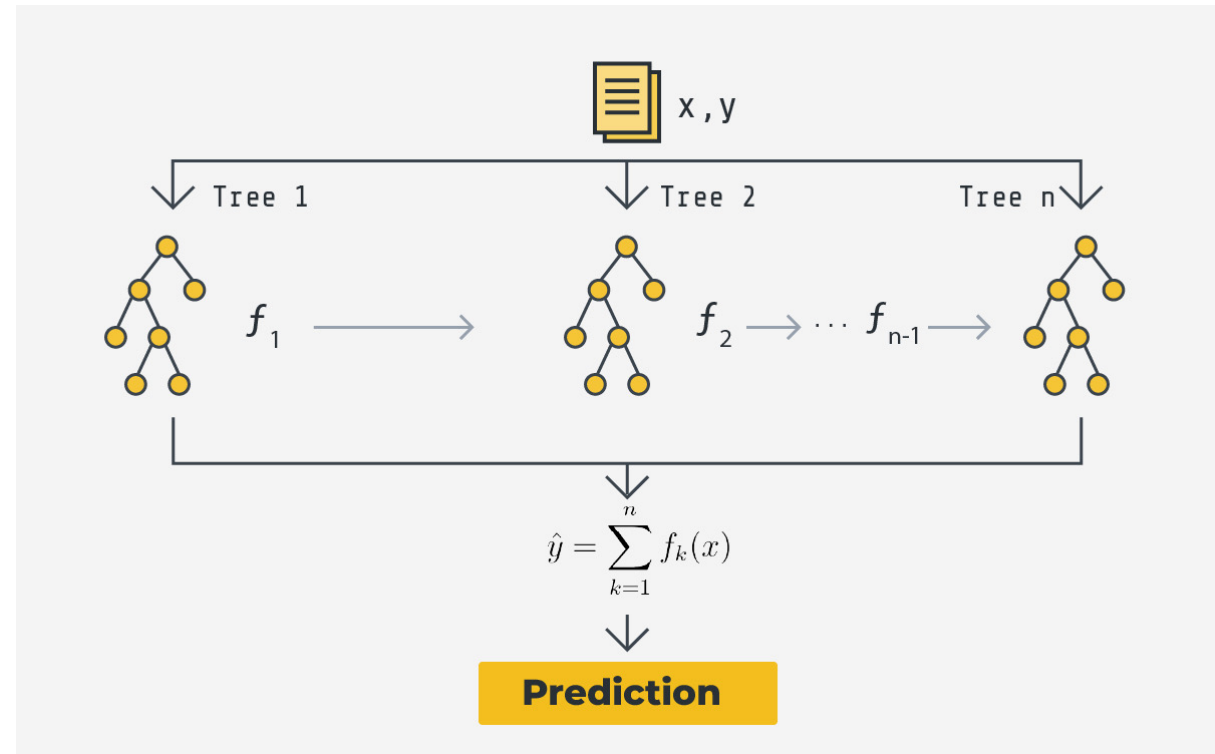


Chance of Successful Decode versus SNR



Machine-Learning Based Filtering of Unpromising Candidates

- LDPC decoding is one of the most computationally intense steps of the pipeline.
- **Question:** Can we filter out candidates that won't decode into anything, without accidentally filtering out candidates that *will* decode into real messages?
- WSJTX has a heuristic for this, and it is already pretty good, but if we can do better, we could significantly cut down wasted CPU decode effort



Prediction Definition: Given SNR, "sync score", number of parity bit errors, etc., predict "Will this candidate, if we apply LDPC, actually decode into a valid FT8 message?"

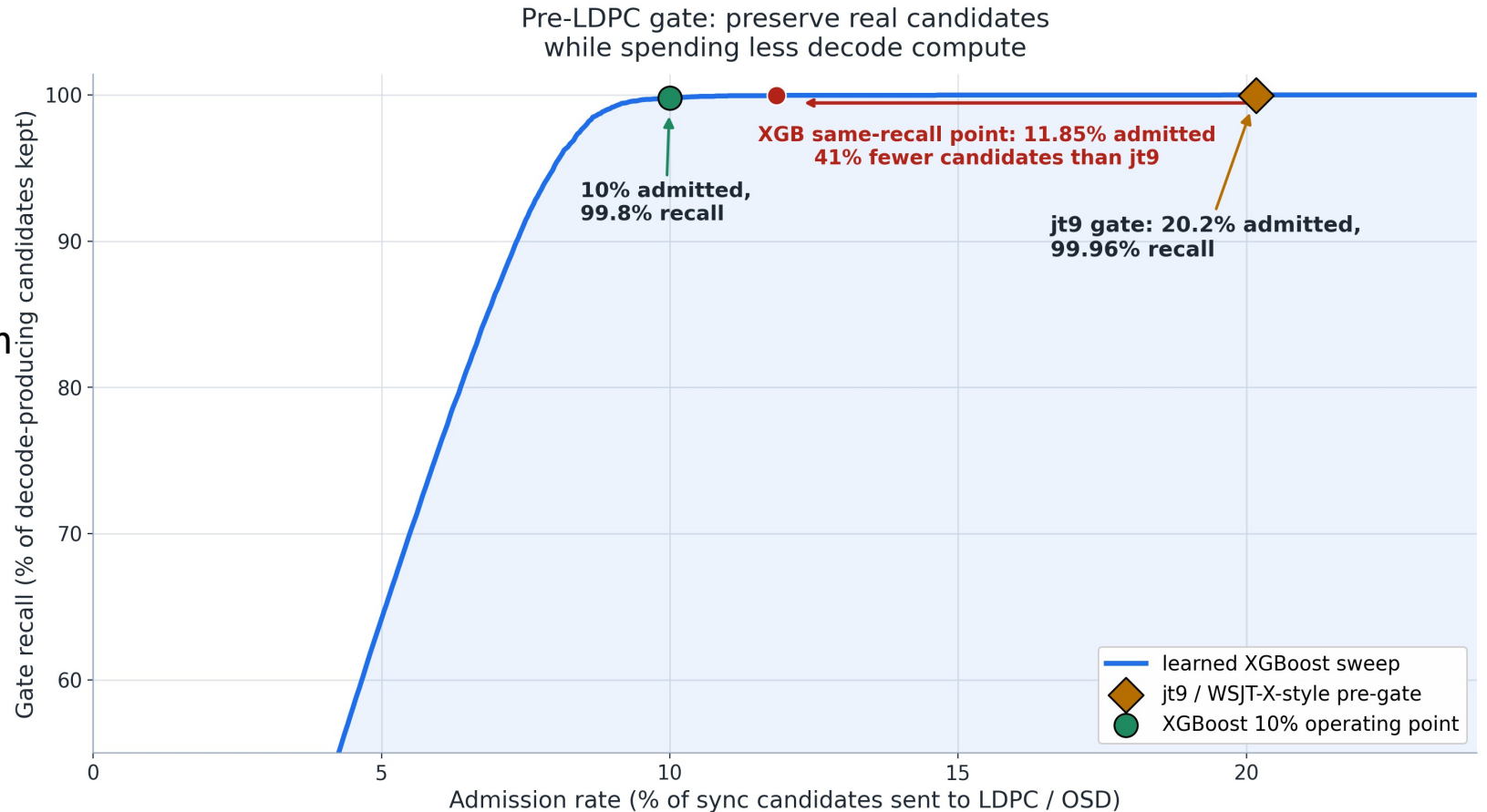
Machine-Learning Based Filtering of Unpromising Candidates

Goal: Given SNR, "sync score", number of parity bit errors, etc., predict *"Will this candidate, if we apply LDPC, actually decode into a valid FT8 message?"*

Approach: Train the XGBoost ML algorithm using decode data from over 2000 WAV files of real HF band conditions

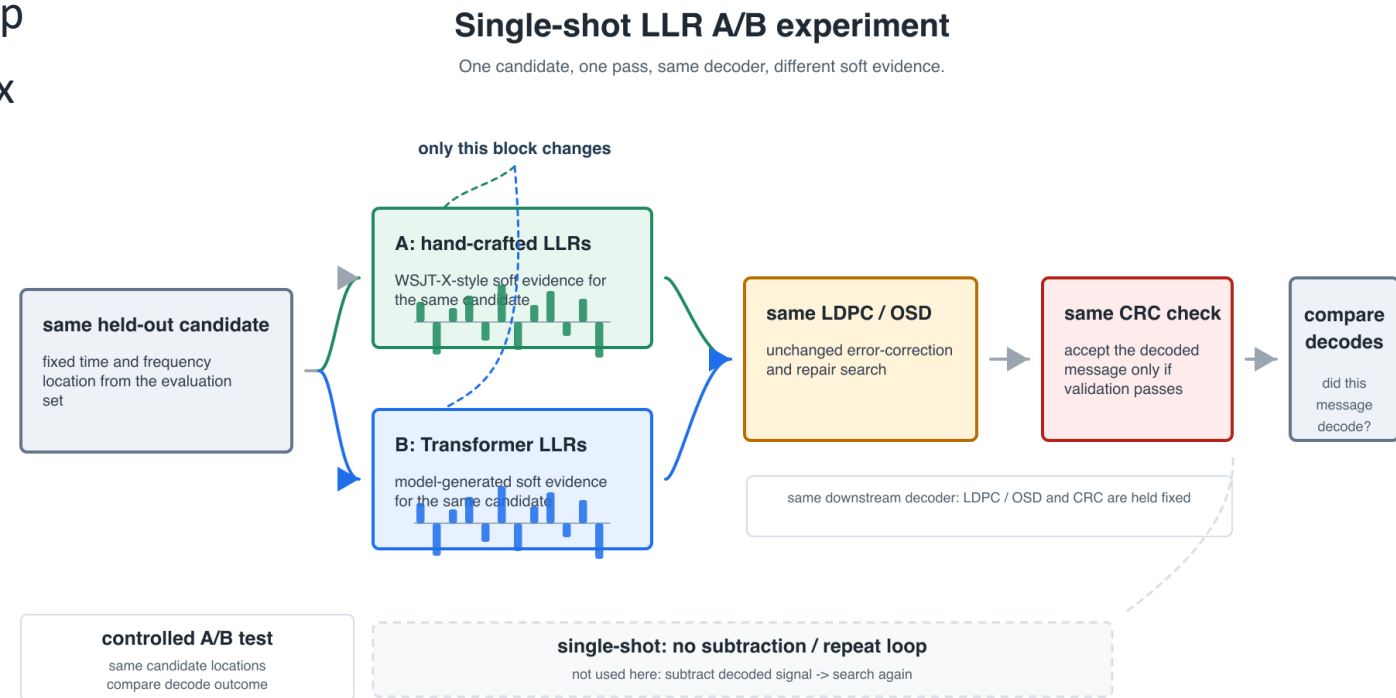
Existing Heuristic: Lets through **20.2%** of "candidates" from the sync pipeline stage. **99.96%** of real decodes are let through

ML Result: Only **11.85%** of candidates let through (**41% fewer**), with **no change in recall of real decodes**.



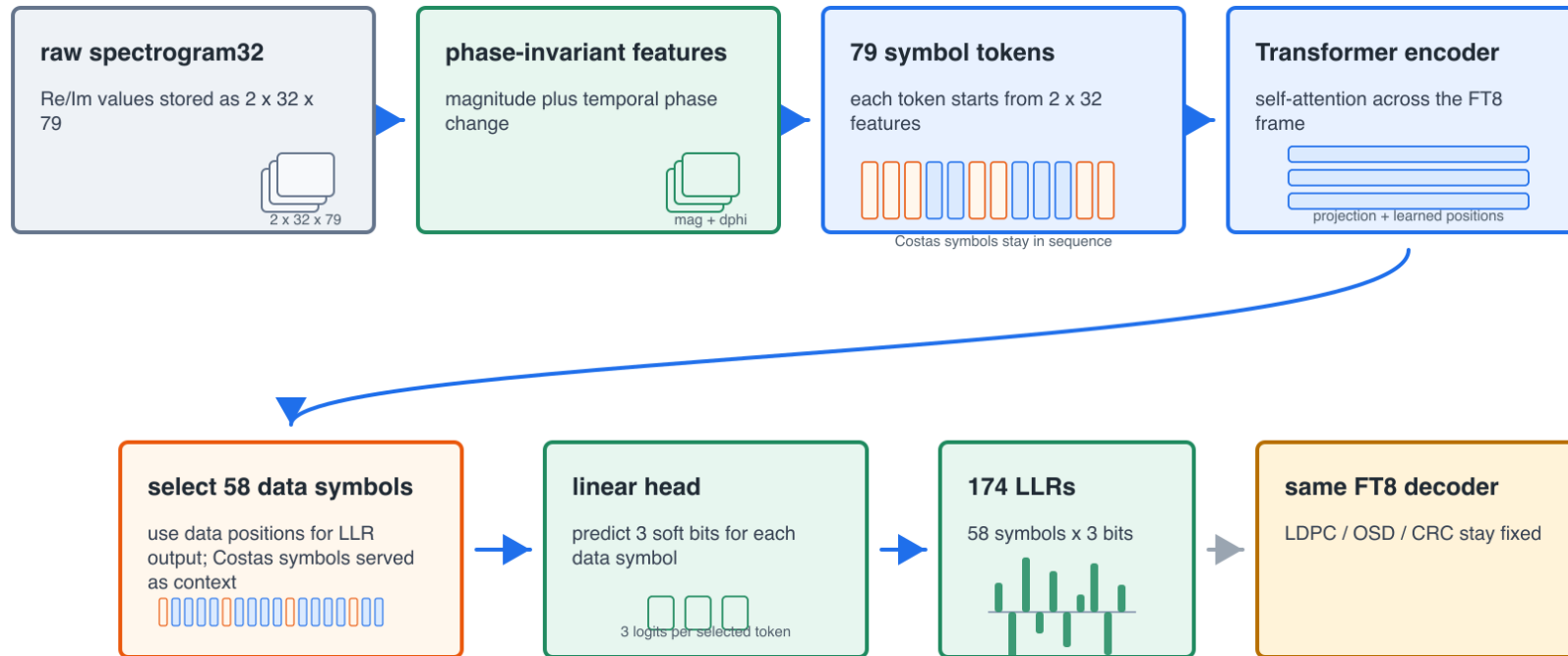
The Single-Shot Question

- Full FT8 decoding tries many variations of the “soft demodulation” LLR conversion, and does up to 9 separate “subtraction” passes. Very complex – hard to parse through the code.
- **One** sync candidate signal can have over 200+ attempted LDPC decodes/CRC checks in a single run
- **Question:** Can we design a LLR calculation function we think is better, and compare in a single “pass” conditions?
- **Approach:** Design several different neural networks to see if we can predict the log-likelihood ratios of the 174 bits more accurately.



Controlled A/B test: same candidate, same decoder, different LLR source.

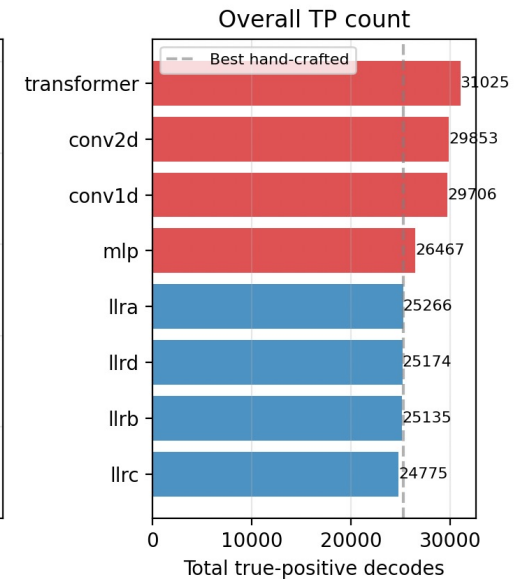
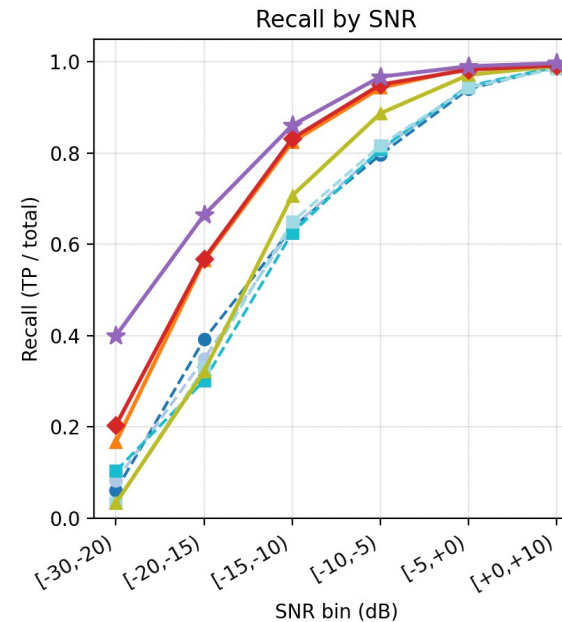
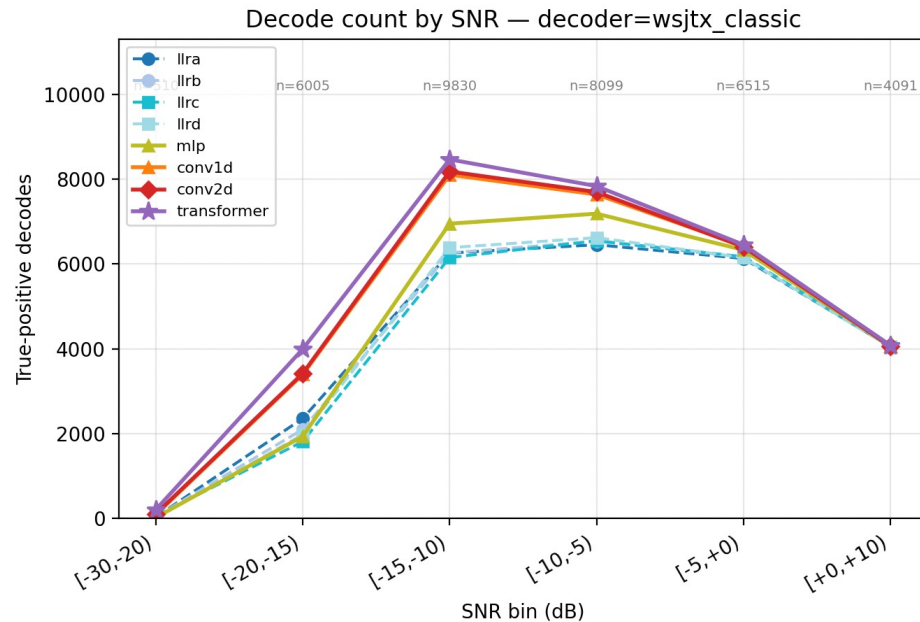
Neural Network: What The Model Sees



- Input: Complex-valued FFT frequency spectrum for *all* 79 symbols (12.64s)
Model: small Transformer over the 79-symbol sequence
- Output: 174 LLRs
- Decoder: unchanged LDPC / OSD / CRC

Result: Biggest Sensitivity Gain Where FT8 Is Hardest

FT8 Decode Counts: Hand-crafted vs Neural Network LLRs (decoder=wsjtx_classic)



- Benchmark is on a **single pass** – no subtraction – of WSJTX hand-crafted LLRs (**llra/llrb/llrc/llrd**) vs neural network models (**mlp, conv1d, conv2d, transformer**) . Best hand-crafted LLR: **25,266** decodes vs Transformer LLR: **31,025** decodes **(+23% more decodes)**
- Weakest [-30, -20] SNR bin: **53/510 -> 204/510 (10% → 40% recall)**
- Second Weakest [-20, -15] SNR bin: **2353/6005 -> 3991/6005 (39% → 66% recall)**

The neural LLR model sees a wider slice of the received signal than a hand-written per-symbol estimator.

That lets it use context: neighboring symbols, off-tone energy, interference from nearby FT8 signals, and small timing or frequency errors.

Conclusion -- More Signal From The Same Noise

- A big part of the magic of FT8 is because every layer squeezes information out of something that almost looks random.
- WSJT-X is already an impressive piece of weak-signal engineering.
- Building ft8lib helped me understand the stack deeply enough to ask where there is still headroom.
- Faster & more sophisticated decoders, along with ML-assisted “filters”, make deeper decoding practical inside the 2.4s limit.
- More accurate probabilistic estimates of bits → a promising pathway to more decodes

